Visitor Management - Backend (Simple Guide)

> *This document explains what the Visitor Management backend does, and how its data is organized, in plain words. For the full API details, open Swagger (see § 5).*

Project: Sustainable Farm - dried mango farm, Burkina Faso -> Germany (Infineon × BIT Excellence Program) Module: Visitor Management (developed by Alix Carine VEBAMBA) Built with: opencode (AI assistant)

1. In a nutshell

The backend covers the full visitor journey: choosing a visit slot, registration, email confirmation, safety briefing, guided tour, feedback - plus events. It is an internal tool for the farm team (guides, front desk), not a public website.

- Technology: Spring Boot (Java 21), PostgreSQL database, REST API.
- Reliability: 229 automated tests, 0 failures.
- Emails: booking confirmation and the 24 h reminder are really sent (SMTP), or
- simply printed to the console during development.
- Frontend-ready: the API accepts calls from the web app (CORS) and ships with
- sample data to start developing immediately (dev profile).

2. What the module does (the screens)

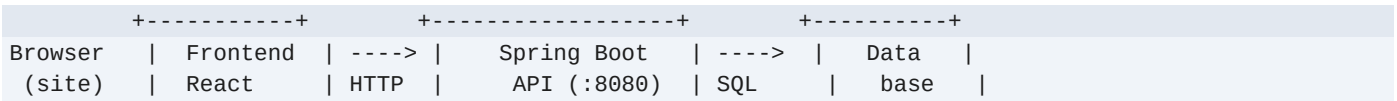
#	Feature	In one sentence
1	Dashboard	Overview: this week's visitors, occupied slots, pending briefings, satisfaction, upcoming events.
2	Farm tour scheduling	Staff creates slots (e.g. 09:00-11:00, 14:00-16:00), max capacity 10 people, closed on Sundays.
3	Visitor registration	Registration form (name, contact, language, special needs?) + tracking: pending -> confirmed -> check-in. Flags prospects (visitors with purchase intent).
4	Educational program	The standard tour path: 6 stops (welcome -> orchard -> irrigation -> solar -> processing -> questions). Workshops (solar, mango tasting, school days) management.
5	Safety briefing	Every confirmed visit has a mandatory safety briefing (who, when, signature). Required before site access.
6	Agritourism bookings	Paid activities (tour 5,000 F, tasting 3,000 F?) with booking, payment, email confirmation and an automatic 24 h reminder.
7	Feedback	After the visit: satisfaction rating, opinion on the briefing, the educational value, recommendation + sending surveys.
8	Events	Group visitors: open days, buyer visits, schools?

In short, the visitor lifecycle:

```
Choose a slot -> Register -> Email confirmation -> Safety briefing
   -> Tour (educational program) -> Feedback -> (optional event)
```

3. How it is organized

One backend, one database. The (upcoming) React frontend will connect to it.



```
         +-----------+        +------------------+        | Postgres |
                                    |                    +----------+
                                    | email (SMTP) / console
                                    v
                            Confirmation + 24 h reminder
```

- All features live in the Visitor Management module
- (modules/visitormanagement/); shared plumbing is in core/.
- The API browser (Swagger) is generated automatically - the best way to map each
- screen -> each URL -> each piece of data.

4. The data (schema & class diagrams)

How to read the diagrams:

```
   One A relates to many B :          One A relates to one B :
+----------+ 1  *  +----------+      +--------+ 1 1 +--------+
|    A     |-------->|    B     |      |   A    |---->|   B    |
+----------+         +----------+      +--------+     +--------+
```

4.1 Scheduling, registration, briefing

The core: who comes, when, and who received the safety briefing.

```
+---------------------+   +----------------------+   +----------------------+
|       TimeSlot      | 1 |     Registration     | 1 |      Briefing        |
|  (tour time slot)   |-->|  (registration)      |-->|  (safety briefing)   |
+---------------------+   +----------------------+   +----------------------+
| id                  |   | id                   |   | id                   |
| date                |   | visitor      (FK)    |   | registration (FK)    |
| startTime           |   | timeSlot     (FK)    |   | deliveredAt          |
| endTime             |   | eventId      (FK)    |   | staffMember          |
| maxCapacity (10)    |   | visitPurpose         |   | signature            |
| status              |   | isProspect (prospect)|   | status               |
| guideId             |   | status               |   +----------------------+
+---------------------+   +----------------------+
        1|                              |
         *|                              | 1..*
+---------------------+                  |
|       Visitor       |<-----------------+
| (visitor / group)   |
+---------------------+
| id                  |
| fullName            |   Relationships :
| groupSize           |   ? 1 slot -> many registrations
| email / phone       |   ? 1 visitor -> many registrations
| language            |   ? 1 registration -> 1 briefing
| type (individual /  |
|   group / school /  |
|   partner)          |
| specialNeeds        |
+---------------------+
```

4.2 Agritourism bookings

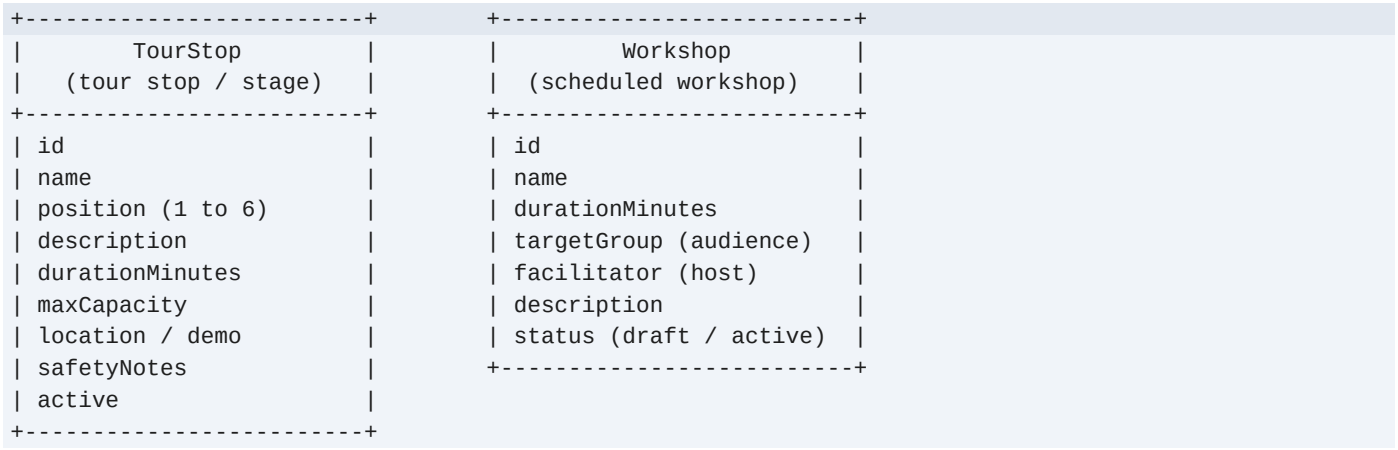
The paid-activity catalog and the bookings, with their emails.

```
+----------------------+   +-------------------------+
|      AgriActivity    | 1 |        Booking          |
| (offered activity)   |-->| (booking)               |
+----------------------+   +-------------------------+
| id                   |   | id                      |
| name                 |   | activity      (FK)      |
| price (FCFA)         |   | timeSlot      (FK)      |
| capacity             |   | visitorFullName         |
```

A booking also reserves a slot of the day (the TimeSlot from 4.1).

4.3 Educational program

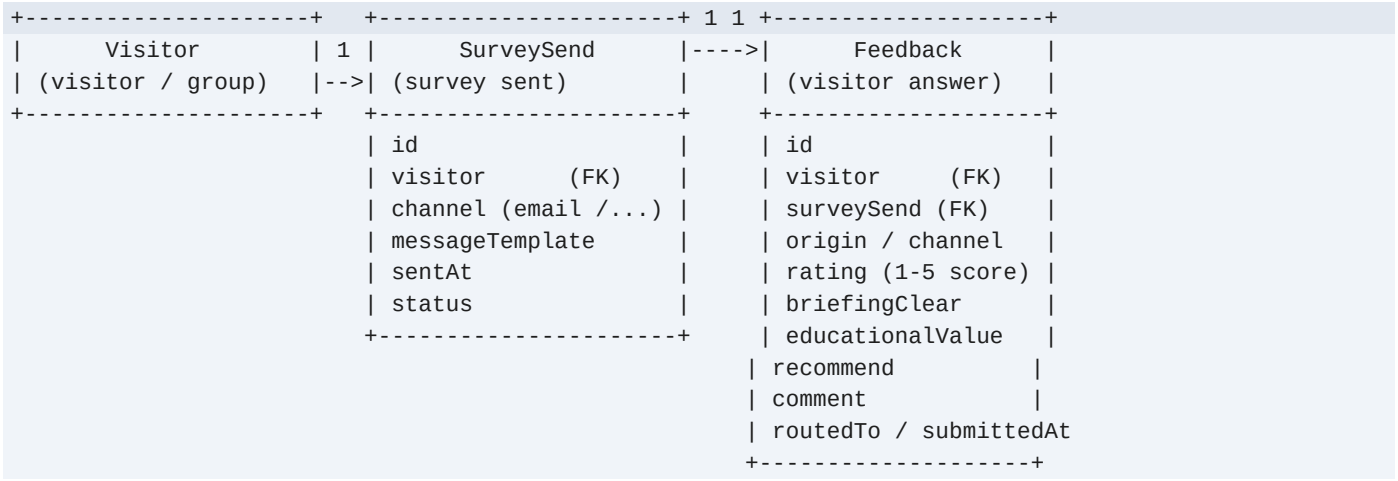
The tour path and the workshops. Two independent lists (no link between them).



The current 6 stops: 1) Welcome & briefing 2) Mango orchard 3) Drip irrigation 4) Solar plant & tracking 5) Processing unit 6) Wrap-up & questions.

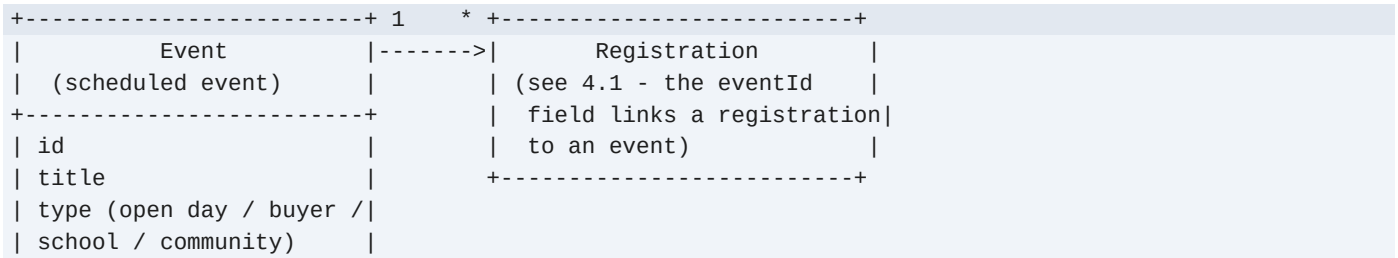
4.4 Feedback and surveys

The visitor answers a survey (SurveySend), which produces a feedback entry.



4.5 Events

An event groups several registrations (hence the eventId field in Registration).



```
| startDateTime / end     |
| maxCapacity             |
| location / description  |
| status (draft/ published|
| / cancelled / ...)      |
+-------------------------+
```

4.6 Relationships summary

Entity	Role	Links
TimeSlot	Tour slot (date/time/capacity)	-> many Registration and Booking
Visitor	Visitor or group	-> many Registration, Feedback, SurveySend
Registration	Sign-up for a slot (or event)	-> 1 Visitor, 1 TimeSlot, 1 Briefing, optional Event
Briefing	Proof of the safety briefing	-> 1 Registration
Event	Event grouping visitors	-> many Registration
AgriActivity	Paid activity in the catalog	-> many Booking
Booking	Activity reservation + emails	-> 1 AgriActivity, 1 TimeSlot
TourStop	Tour path stage	independent
Workshop	(Educational) workshop	independent
SurveySend	Survey sent to the visitor	-> 1 Visitor, -> 1 Feedback
Feedback	Visitor's answer	-> 1 Visitor, 1 SurveySend

Every entity shares an id and the timestamps createdAt / updatedAt.

5. Accessing the API (Swagger)

The best entry point is Swagger, generated automatically.

- Swagger UI: http://localhost:8080/swagger-ui/index.html
- Health check: http://localhost:8080/actuator/health

All endpoints live under /api/v1:

Resource	Main URLs
Dashboard	/dashboard (KPIs of the week + upcoming tasks)
Time slots	/time-slots (+ /availability)
Visitors	/visitors
Registrations	/registrations (+ /approve, /reject, /check-in, /cancel)
Briefings	/registrations/{id}/briefing (+ /deliver)
Educational program	/tour-stops, /workshops (+ /publish, /deactivate)
Bookings	/activities, /bookings (+ /pay, /confirm, /complete, /cancel)
Feedback	/feedback, /feedback/summary, /surveys
Events	/events (+ /register, /publish, /registrations)

6. Getting started

Prerequisite: Docker. Copy .env.example to .env, then:

```
docker compose up -d --build
# Backend :  http://localhost:8080
# Swagger :  http://localhost:8080/swagger-ui/index.html
```

Useful options (in .env)

Variable	Purpose	Example
SPRING_PROFILES_ACTIVE	Loads sample data (stops, workshops, activities, slots, one event and one visitor) for development. Never in production.	dev
APP_CORS_ORIGINS	Allows the web app (frontend) to call the API.	http://localhost:3000, http://localhost:5173
MAIL_ENABLED	true -> real email sending (SMTP); false -> prints to console.	false
MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM	SMTP server settings.	smtp.orange.bf / 587

7. Quality

- 229 automated tests, 0 failures: business rules (capacities, statuses, payments),
- emails (confirmation, 24 h reminder, failed send), all URLs and the data schema.
- Re-run the tests (via Docker, no local JDK required):

```
docker run --rm -v "$PWD/backend":/src -w /src -v "$HOME/.m2":/root/.m2 \
  maven:3.9-eclipse-temurin-21 ./mvnw test
```

8. What's next

The backend is completed and ready. Next: build the frontend (the screens around the 8 features of § 2). As soon as the backend runs with SPRING_PROFILES_ACTIVE=dev, the database contains the sample data, so the site can be developed and tested right away.

Last updated: 2026-09-10